Unit-01

What is Python

Python is a **high-level, interpreted and Object-Oriented Programming (OOPs)** and known for its simplicity, readability, and flexibility. It emphasizes code readability with its **easy-to-learn syntax**, making it accessible for beginners and powerful enough for experts.It is widely used in **web development, data science, machine learning, AI, scientific computing,** and more.

History of Python

- **Creator:** Python was created by **Guido van Rossum**.
- **First Release:** Python 1.0 was released in **1991**.
- **Inspiration:** Guido was inspired by the **ABC programming language (WRITE "Hello, World!")**.
- The name "Python" was chosen as a tribute to the British comedy series **"Monty Python's Flying Circus,"** not the snake.

Why Python popular?

1. **Ease of Use:** Simplify programming for beginners and make it easy to write code.
2. Python is used in **web development** (Django, Flask), **data science**, **automation**, **game development**, **desktop applications**, etc.
3. **Readability:** Encourage clean and maintainable code that is easy to understand and debug.
4. **Extensibility:** Allow integration with other languages like C, C++, and Java.
5. **Cross-Platform Compatibility:** Python runs on Windows, macOS, Linux, and more.
6. **Rich Ecosystem and Libraries:** Extensive libraries such as **NumPy, Pandas, Matplotlib**, and **TensorFlow** cater to diverse domains like data analysis, machine learning, and AI.
7. **Open-Source:** Python is free to use, modify, and distribute.

Drawbacks of Python

1. **Slower Execution Speed:** As an interpreted language, Python is slower than compiled languages like C++ or Java.
2. **Memory-Intensive:** Python's dynamic nature can lead to higher memory consumption.

3. **Not Ideal for Mobile Development:** Python is not commonly used for mobile application development due to its slower runtime.

Applications of Python

1. **Web Development:**
   - Frameworks like **Django** and **Flask** are used to build scalable web applications.
2. **Data Science and Machine Learning:**
   - Python dominates in this field with libraries like **Pandas, Scikit-learn, TensorFlow**, and **PyTorch**.
3. **Game Development:**
   - Python is used with engines like **Pygame** for simple game development.
4. **Scientific Computing:**
   - Libraries like **SciPy** and **NumPy** make Python a popular choice for researchers.
5. **Artificial Intelligence and Deep Learning:**
   - Python is the leading language in AI and deep learning.

First Python Program

```
print("Hello, World!")
```

Output:

```
Hello, World!
```

Comparison Between Compiler and Interpreter

Feature	Compiler	Interpreter
Translation	Translates the entire program at once.	Translates and executes line by line.
Execution	Executes the program after full translation.	Executes the program immediately after translation of each line.
Error Detection	Detects all errors during the compilation process.	Stops at the first error encountered.
Execution Speed	Faster execution after compilation.	Slower execution since translation happens during execution.
Memory Usage	Typically uses more memory (due to generating an executable).	Uses less memory but requires the interpreter each time.

Feature	Compiler	Interpreter
Portability	Less portable because the compiled machine code is platform-dependent.	More portable because the interpreter is platform-dependent, not the source code.

Variable

A **variable** is a name given to a memory location where data is stored. In Python, variables are used to hold values like numbers, strings, lists, etc. Unlike some programming languages, you don't need to declare the type of a variable in Python; it is dynamically inferred based on the value assigned to it.

Example:

```
x = 10          # Variable x is assigned an integer value
a = 10.5        # Variable x is assigned an float value
name = "Alice" # Variable name is assigned a string value
```

Assigning Values to Variables

1. Multiple Assignments:

```
x, y, z = 1, 2.4, "python"  # Assigns 1 to x, 2.4 to y, and "python" to z
```

3. Assigning the Same Value to Multiple Variables:

```
a = b = c = 0  # All variables are assigned the value 0
```

Rules for Naming Variables in Python

1. Variable Names Must Start with a Letter or an Underscore (_):

Valid:

```
my_var = 10
_my_var = 20
```

Invalid:

```
1var = 30  # SyntaxError: cannot start with a digit
```

2. Variable Names Can Only Contain Letters, Numbers, and Underscores (_):

Valid:

```
var_1 = 15
my_variable_123 = "Hello"
```

Invalid:

```
my-variable = 50   # SyntaxError: contains a hyphen
var$ = 100         # SyntaxError: contains a special character
```

3. Variable Names Are Case-Sensitive:

```
name = "Alice"
Name = "Bob"
print(name)   # Output: Alice
print(Name)   # Output: Bob
```

4. Variables Cannot Be a Keyword or Reserved Word:

Keywords like `if`, `else`, `for`, `while`, `class`, and `def` cannot be used as variable names.

```
if = 10   # SyntaxError
```

`input()` in Python

The `input()` function reads a line from the user, converts it to , and stores it in a variable. By default, everything entered via `input()` is treated as a string.

Syntax: `variable = input("Prompt message: ")`

Optional text displayed to the user before taking input.

1. Basic Example

```
name = input("Enter your name: ")
print("Hello, ",name )
```

Output:

```
Enter your name: Alice
Hello,Alice
```

2. Taking Numerical Input

Since `input()` always returns a string, you need to convert the input to a numeric type like `int` or `float`.

```
age = int(input("Enter your age: "))
print("Your age:", age)
```

Output:

```
Enter your age: 26
Your age: 26
```

3. Taking Multiple Inputs

a) space-separated inputs using `split()`

```
x, y = input("Enter two numbers separated by space: ").split()
print("You entered:", x, "and", y)
```

Output:

```
Enter two numbers separated by space: 10 20
You entered: 10 and 20
```

b) With `map()` for type conversion

```
a, b = map(int, input("Enter two integers: ").split())
print("Sum is:", a + b)
```

Output:

```
Enter two integers: 5 7
Sum is: 12
```

c) Unpacking multiple types (str, int, float)

```
name, age, city = input("Enter name, age, city: ").split()
age = int(age)
city = str(city)
print(f"Name: {name}, Age: {age}, City: {city}")
```

Output:

```
Enter name, age, salary: Asif 26 Firozabad
Name: Asif, Age: 26, City: Firozabad
```

Practice Question

1) Write a program to calculate the area of a square. Ask the user to input the side length of the square.
2) Write a program to calculate the area of a rectangle. Ask the user to input the length and width.
3) Write a program to calculate the area of a circle. Ask the user to input the radius (use $\pi = 3.14$).
4) Write a program to calculate the perimeter of a square. Ask the user for the side length.
5) Write a program to calculate the perimeter of a rectangle. Ask the user for the length and width.
6) Write a program to calculate the square of a number. Ask the user for the number.
7) Write a program to calculate the cube of a number. Ask the user for the number.
8) Write a program to convert Celsius to Fahrenheit. Use the formula: $F = (C * 9/5) + 32$

9) Write a program to ask the user for two numbers and make basic calculator.
10) Write a program to calculate the simple interest. Ask the user for the principal amount, rate of interest, and time.
11) Write a program to ask the user for two numbers and swap their values. Print the numbers before and after swapping.

Use `type()`?

The **`type()` function** in Python is used to determine the **data type** of a value or variable. It returns the type of the given object

1. To determine the type of a variable or object.
2. To validate user input or data types in programs.
3. To understand dynamic typing in Python.

Examples of `type()` Function

```
# Integer
x = 42
print(type(x))  # Output: <class 'int'>

# Float
y = 3.14
print(type(y))  # Output: <class 'float'>

# String
name = "Alice"
print(type(name))  # Output: <class 'str'>

# Boolean
is_active = True
print(type(is_active))  # Output: <class 'bool'>
```

---

3. Type Conversion in Python

Example:

```
# Convert integer to float
num = 10
float_num = float(num)
print(type(float_num))  # Output: <class 'float'>

# Convert float to integer
price = 12.99
```

```
int_price = int(price)
print(type(int_price))  # Output: <class 'int'>

# Convert string to integer
s = "100"
num = int(s)
print(type(num))  # Output: <class 'int'>
```

Keywords in Python

keywords are reserved words that have special meaning and are used to define the syntax of the language. Keywords cannot be used as variable names, function names, or any other identifiers. They are part of the core Python language and are predefined in the language.

List of Python Keywords

```
import keyword
print(keyword.kwlist)
```

Some of the most commonly used Python keywords include:

- `False`: Boolean value, representing false.
- `None`: Represents the absence of a value.
- `True`: Boolean value, representing true.
- `and`: Logical operator, used for logical conjunction.
- `or`: Logical operator, used for logical disjunction.
- `not`: Logical operator, used for logical negation.
- `if`: Used for conditional statements.
- `else`: Used in conditional statements when the "if" condition fails.
- `elif`: Used to check additional conditions in an `if` statement.
- `for`: Used to create loops for iterating over a sequence.
- `while`: Used to create loops that continue while a condition is true.
- `break`: Used to exit a loop.
- `continue`: Used to skip the current iteration of a loop.
- `def`: Used to define a function.
- `class`: Used to define a class.
- `try`: Used to start a block of code to handle exceptions.
- `except`: Defines the block of code to handle exceptions raised in `try` block.
- `import`: Used to import a module or package.
- `return`: Used to return a value from a function.
- `in`: Used to check if a value exists in a sequence (like a list).
- `is`: Used to test object identity.

ASCII

ASCII stands for **American Standard Code for Information Interchange**.
It is a character encoding standard used to represent **text in computers**, communication equipment, and other devices that use text.

- To convert characters (letters, digits, symbols) into **numeric codes** that computers can store and manipulate.
- Each character (like `'A'`, `'1'`, `'@'`) is mapped to a **unique number** between **0 and 127**.

Why is ASCII Important?

- It was one of the first **universal standards** for encoding text.
- Ensures **compatibility** between different systems and devices.
- Still forms the basis of **modern encodings** like **UTF-8**, where the first 128 characters are identical to ASCII.

ASCII Value Range

Range	Description
0–31	Control characters (non-printable)
32–126	Printable characters (letters, digits, etc)
127	DEL (delete control character)

Examples of ASCII Codes

Character	ASCII Code
`'A'`	65
`'a'`	97
`'0'`	48
`' '`	32 (space)
`'@'`	64
`'\n'`	10 (newline)

ASCII in Python

✅ `ord()` function

Returns the **ASCII code** of a character.

```
print(ord('A'))  # Output: 65
print(ord('a'))  # Output: 97
```

✅ `chr()` function

Returns the **character** corresponding to an ASCII code.

```
print(chr(65))   # Output: 'A'
print(chr(97))   # Output: 'a'
```

ASCII vs Unicode

Feature	ASCII	Unicode
Size	7-bit (128 characters)	8, 16, or 32-bit (1.1 million+)
Language Support	English only	Supports **all languages**
Modern Usage	Limited use	Widely used (UTF-8/16/32)

Comments in Python

Comments in Python are used to explain the code and make it more readable. They are ignored by the Python interpreter during execution. You can add comments to your Python code to describe what specific parts of the code are doing, making it easier for others (or yourself) to understand the logic.

There are two main types of comments in Python:

1. **Single-line Comments**
2. **Multi-line Comments**

1. Single-line Comments

A single-line comment starts with a hash (`#`) symbol. Anything following the `#` on that line is considered a comment and is not executed by Python.

Example:

```
x = 5  # This is also a comment after a statement
```

2. Multi-line Comments

Multi-line comments are useful when you want to write a comment that spans multiple lines. There are two ways to write multi-line comments in Python:

1. **Using Triple Quotes (`'''` or `"""`)**: Triple quotes (`'''` or `"""`) can be used to create multi-line comments, but technically, these are multi-line strings. Since they are not assigned to a variable or used in an expression, they act as comments.

   Example:

   ```
   '''
   This is a multi-line comment
   using triple single quotes.
   It spans multiple lines.
   '''
   ```

   Or using double quotes:

   ```
   """
   This is also a multi-line comment
   using triple double quotes.
   It also spans multiple lines.
   """
   ```

---

1. Indentation in Python

indentation is used to define the structure of code blocks, such as those within loops, conditional statements, functions, and classes. Unlike other programming languages that use braces (`{}`) or keywords (e.g., `end`), Python uses whitespace indentation to indicate blocks of code (Use **4 spaces per indentation level**, as per the **PEP 8** style guide).

Incorrect Indentation Example:

```
if True:
print("This will throw an error")  # IndentationError
```

Correct Indentation Example:

```
if True:
    print("This is properly indented")
```

How Indentation Works:

1. Conditional Statements

```
x = 10
if x > 5:
    print("x is greater than 5")  # Indented block
else:
    print("x is less than or equal to 5")
```

2. Nested Blocks

Indentation depth increases with each nested block.

```
x = 10
if x > 5:
    if x < 20:
        print("x is between 5 and 20")  # Nested block
```

Common Indentation Errors

1. **Mixing Tabs and Spaces**:

   Example:

   ```
   if True:
       print("Using spaces")
    print("Using tabs")  # TabError
   ```

2. **Unintended Indentation**:

   Example:

   ```
   print("Hello")
       print("This line has unintended indentation")  # IndentationError
   ```

---

2. Naming Conventions in Python

Standard Naming Conventions:

1. Variable Names

- Use **lowercase letters** with words separated by underscores.
- Keep names descriptive and meaningful.

Examples:

```
age = 25
first_name = "Alice"
total_price = 99.99
```

---

2. Function Names

- Use **lowercase letters** with words separated by underscores.
- Function names should describe their purpose.

Examples:

```
def calculate_sum(a, b):
    return a + b

def greet_user(name):
    print(f"Hello, {name}")
```

---

3. Class Names

- Use **CamelCase** (also called PascalCase).
- Start each word with an uppercase letter.

Examples:

```
class EmployeeDetails:
    pass
```

---

4. Constant Names

- Use **ALL_CAPS** with words separated by underscores.

Examples:

```
PI = 3.14159
MAX_LIMIT = 100
DEFAULT_NAME = "Guest"
```

---

5. Special Variables

- Use a single or double leading underscore for special purposes.
    - **Single Leading Underscore (`_variable`)**: Indicates a variable is intended for internal use.
    - **Double Leading Underscore (`__variable`)**: Triggers name-mangling to prevent conflicts in subclasses.
    - **Single Underscore (`_`)**: Often used as a throwaway variable.

Examples:

```
_private_variable = "Internal use only"

class MyClass:
    def __init__(self):
        self.__hidden_variable = "Name mangling example"
```

---

6. Naming Boolean Variables

- Use names that indicate `True`/`False` values.
- Prefix with `is`, `has`, `can`, or `should`.

Examples:

```
is_active = True
has_permission = False
should_restart = True
```

Cases

1. Camel Case (camelCase)

- In camel case, the first word starts with a lowercase letter, and subsequent words start with uppercase letters.
- Used in variable names and function names in some programming languages (e.g., JavaScript).
- **Example:**
- `myVariableName = "Camel Case Example"`
- `getUserName = lambda: "John Doe"`

---

2. Pascal Case (PascalCase)

- Similar to camel case, but the first letter of the first word is also uppercase.
- Commonly used for class names in Python.
- **Example:**
- `class UserProfile:`
- `    pass`
- `class MovieRecommendationSystem:`
- `    pass`

---

3. Snake Case (snake_case)

- Words are separated by underscores (_), and all letters are lowercase.
- Commonly used for function names, variable names, and file names in Python.
- **Example:**
- `user_name = "John Doe"`
- `def get_movie_recommendations():`
- `    pass`

---

4. Screaming Snake Case (SCREAMING_SNAKE_CASE)

- Similar to snake case but with all uppercase letters.
- Used for constants in Python.
- **Example:**
- `MAX_USERS = 100`
- `API_KEY = "12345XYZ"`

---

5. Kebab Case (kebab-case)

- Words are separated by hyphens (-).
- **Not used in Python** because variable names cannot contain hyphens.
- Commonly used in URLs and filenames.
- **Example (in a URL or config file):**
- `my-profile-page`

---

6. Upper Camel Case (UpperCamelCase)

- Another term for Pascal Case.
- Used for class names in Python.
- **Example:**
- `class DataScienceModel:`
- `    pass`

---

7. Lower Camel Case (lowerCamelCase)

- Another term for Camel Case.
- Used in some programming languages for variable and function names, but Python prefers snake_case.
- **Example:**
- `def getUserDetails():`
- `    pass`

Python Naming Conventions (PEP 8 Guidelines)

- **Variables & Function Names:** `snake_case`
- **Class Names:** `PascalCase`
- **Constants:** `SCREAMING_SNAKE_CASE`
- **Private Variables:** `_single_leading_underscore`
- **Strongly Private Variables:** `__double_leading_underscore`
- **Magic Methods (Dunder Methods):** `__init__`, `__str__`, etc.

Data Types in Python

1. Numeric Types

`int` – Integer Numbers

- Whole numbers (positive, negative, or zero) without decimals.
- Arbitrary precision: Python supports very large integers without overflow.

Example:

```
x = 10
print(type(x))  # <class 'int'>
```

More Examples:

```
a = -100
b = 0
c = 12345678901234567890  # Very large integer
```

Common Integer Operations:

```
x = 7
y = 3

print(x + y)  # 10
print(x - y)  # 4
print(x * y)  # 21
print(x // y) # 2 (Floor Division)
print(x % y)  # 1 (Modulus)
print(x ** y) # 343 (Exponentiation)
```

`float` – Floating Point Numbers

- Numbers with decimal points.
- Approximation of real numbers.
- IEEE 754 double-precision floating-point format.

Example:

```
y = 3.14
print(type(y))  # <class 'float'>
```

More Examples:

```
a = -7.5
```

```
b = 0.0
c = 1.6e3  # Scientific notation → 1600.0
```

Common Float Operations:

```
a = 7.0
b = 2.0

print(a / b)  # 3.5
print(round(3.14159, 2))  # 3.14
```

`complex` – Complex Numbers

- Used to represent complex numbers (real + imaginary part).
- Syntax: `a + bj` (use `j` for the imaginary unit).
- `complex` is a built-in Python type.

Example:

```
z = 2 + 3j
print(type(z))  # <class 'complex'>
```

Accessing Parts:

```
print(z.real)  # 2.0 → Real part
print(z.imag)  # 3.0 → Imaginary part
```

More Examples:

```
a = complex(5, -2)  # 5 - 2j
b = 1 + 0j
```

Use Cases:

- Engineering and scientific calculations
- Signal processing, control systems, quantum computing

Operations with Complex Numbers:

```
a = 1 + 2j
b = 2 + 3j

print(a + b)       # (3+5j)
print(a * b)       # (-4+7j)
print(abs(a))      # √(1² + 2²) = 2.236
```

---

2. Text Type

str-String

A **string** in Python is a **sequence of Unicode characters** enclosed within single (`'...'`), double (`"..."`), or triple quotes (`'''...'''` or `"""..."""`).

```
name = "Asif"
print(type(name))  # <class 'str'>
```

Creating Strings

```
s1 = 'Hello'
s2 = "World"
s3 = """This is a
multi-line string."""
```

Why Use ' ', " and """ in Python?

Python allows you to use **single quotes** (`'`), **double quotes** (`"`), and **triple quotes** (`"""` or `'''`) to define strings. The choice often depends on the situation:

1. **Single Quotes (`'`):**
   - Used for simple, short strings without apostrophe (`'`) characters.
   - Example: `'Hello'`
2. **Double Quotes (`"`):**
   - Similar to single quotes but handy when the string without (`"`) characters.
   - Example: `"It's a sunny day"`
3. **Triple Quotes (`"""` or `'''`):**
   - Used for multi-line strings or when the string contains both single and double quotes.
   - Example:

```
1. """This is a
   multi-line string."""
2. """She said, "It's a wonderful day!" """
```

Indexing and Slicing

Indexing:

```
s = "Python"
print(s[0])   # 'P' (first character)
print(s[-1])  # 'n' (last character)
```

Slicing:

```
print(s[1:4])     # 'yth'
print(s[:3])      # 'Pyt'
print(s[::2])     # 'Pto' (every 2nd char)
```

String Immutability

```
s = "Hello"
# s[0] = 'h'  ❌ Error: strings are immutable
# But you can create a new one:
s = 'h' + s[1:]  # 'hello'
```

Using f-strings (Recommended in Python 3.6+)

```
name = "Asif"
age = 26
print(f"My name is {name} and I am {age} years old.")
```

Using `format()`

```
print("Hello, {}!".format("Asif"))
```

Escape Characters

Escape	Meaning	Example
`\n`	New line	`"Hello\nWorld"`
`\t`	Tab space	`"Hello\tWorld"`
`\\`	Backslash	`"C:\\Users\\"`
`\'`	Single quote	`'It\'s fine'`
`\"`	Double quote	`"He said \"Hi\""`

String Operators

Operator	Description	Example
`+`	Concatenation	`"Hello" + "World"`
`*`	Repetition	`"Hi" * 3 → 'HiHiHi'`
`in`	Membership check	`'a' in 'cat' → True`

Expression Output

Question 1: String and Number Multiplication

```
Txt = "a"
result = 4 * Txt + 3 * Txt
```

```
print(result)
```
Output:
```
aaaaaaa
```

Question 2: Concatenation with Multiplication

```
A = "Hello"
B = "World"
result = A+B
print(result)
```
Output:
```
HelloWorld
```

Question 3: Nested Operations

```
Txt = "*"
result = (2 * Txt + 3 * Txt) * 2
print(result)
```
Output:
```
**********
```

Question 4: Combining Strings and Numbers

```
A = "Python"
B = 3
C = "!"
result = (A + C) * B
print(result)
```
Output:
```
Python!Python!Python!
```

4. Sequence Types

a) list → Ordered, mutable collection

```
fruits = ['apple', 'banana', 'cherry']
type(fruits)  # <class 'list'>
```

Supports:

- Indexing & slicing
- Mutability: `fruits[1] = 'orange'`

b) tuple → Ordered, immutable collection

```
coordinates = (10, 20)
type(coordinates)  # <class 'tuple'>
```

Used when data should not change.

c) set → Unordered, unique items

```
s = {1, 2, 3, 2}
type(s)  # <class 'set'>
```

- Duplicate values are removed
- Unordered, no indexing

d) frozenset → Immutable set

```
fs = frozenset([1, 2, 3])
type(fs)  # <class 'frozenset'>
```

e) dict → Key-value pairs

```
person = {'name': 'Asif', 'age': 26}
type(person)  # <class 'dict'>
```

- Access: `person['name']`
- Mutable
- Keys must be unique

5. NoneType → Represents "No Value" or "Null"

`NoneType` is a special **built-in data type** in Python that represents the **absence of a value** or a **null value**.
It has **only one value**: `None`.

```
x = None
print(type(x))  # Output: <class 'NoneType'>
```

Why is None Used?

- As a **default value** for function arguments.
- To indicate **no result** or **no return value**.
- To act as a **placeholder** when a value is not yet assigned.
- For **null checks** in logic flows (`if x is None:`).

Examples and Use Cases:

1. Default Function Argument

```
def greet(name=None):
    if name is None:
        print("Hello, Guest!")
```

```
    else:
        print(f"Hello, {name}!")

greet()            # Output: Hello, Guest!
greet("Asif")      # Output: Hello, Asif!
```

2. Placeholder for Uninitialized Variables

```
user_email = None  # Placeholder before assigning an actual email
```

❌ Common Mistake:

```
# Bad practice
if x == None:        # Might work, but not safe

# Good practice
if x is None:        # Correct and safe
```

Operators:

operators are special symbols used to perform operations on variables and values. They are categorized based on the type of operations they perform.

a. Arithmetic Operators

Operator	Description	Example
`+`	Addition	`3 + 5 = 8`
`-`	Subtraction	`10 - 7 = 3`
`*`	Multiplication	`4 * 2 = 8`
`/`	Division	`15 / 3 = 5.0`
`//`	Floor Division	`17 // 3 = 5`
`%`	Modulus (Remainder)	`17 % 3 = 2`
`**`	Exponentiation (Power)	`3 ** 3 = 27`

b. Comparison (Relational) Operators

These compare two values and return a Boolean (`True` or `False`).

Operator	Description	Example
`==`	Equal to	`5 == 5 → True`
`!=`	Not equal to	`5 != 3 → True`
`<`	Less than	`3 < 5 → True`
`>`	Greater than	`10 > 7 → True`
`<=`	Less than or equal to	`5 <= 6 → True`
`>=`	Greater than or equal to	`7 >= 3 → True`

c. Logical Operators

Used to combine conditional statements.

Operator	Description	Example
`and`	Logical AND (both true)	`True and False → False`
`or`	Logical OR (one true)	`True or False → True`
`not`	Logical NOT (inverts)	`not True → False`

d. Bitwise Operators

Bitwise operators allow you to perform operations **on individual bits** of numbers.
These operators treat values as a **sequence of bits (binary)**, not as decimal numbers.

Practical Use Cases

1. **Low-Level Programming:** Hardware/device driver manipulation.
2. **Performance Tricks:** Fast multiplication/division by powers of 2.
3. **Data Compression:** Bit packing/unpacking.
4. **Cryptography:** XOR is used in simple encryption schemes.
5. **Flags & Masks:** Use AND/OR to turn on/off specific bits (feature flags).

Binary Refresher

Decimal	Binary
5	0101
3	0011
10	1010
12	1100
7	0111

Use `bin(x)` to check:

```
bin(5)   # '0b101'
```

1. Bitwise AND (&)

Returns 1 if **both bits** are 1, else 0.

Example:

```
a = 12        # 1100
b = 5         # 0101

result = a & b
print(result)  # Output: 4
```

2. Bitwise OR (|)

Returns 1 if **at least one** bit is 1.

Example:

```
a = 10        # 1010
b = 4         # 0100

result = a | b
print(result)  # Output: 14
```

3. Bitwise XOR (^)

Returns 1 if the bits are **different**, else 0.

Example:

```
a = 7        # 0111
b = 12       # 1100

result = a ^ b
print(result)  # Output: 11
```

4. Bitwise NOT (~)

Inverts each bit.
`~x` is equal to `-(x + 1)` due to two's complement representation.

Example:

```
a = 10        # 1010

result = ~a
print(result)  # Output: -11
```

5. Left Shift (<<)

Shifts bits to the left by `n` positions.
Removes bits from the left and adds `0`s on the right. This increases the value of the number.

In Mathematical Terms:

$a << n = a \times (2^n)$

- Shifts the bits of `a` to the **left by `n` places**.
- Each shift to the left **doubles** the number.
- So shifting left by `n` positions **multiplies the number by 2^n**.

Example:

```
a = 6        # Binary: 0110

result = a << 2
print(result)  # Output: 24
```

6. Right Shift (>>)

Shifts bits to the right by `n` positions.
Removes bits from the right and **adds 0s (or sign bit) on the left**, depending on the number type (for positive numbers, 0s are added).

In Mathematical Terms:

$a >> n = a \div (2^n)$

- Shifts the bits of `a` to the **right by `n` places**.
- Each shift to the right **halves** the number.
- So shifting right by `n` positions **divides the number by 2^n (integer division)**.

Example:

```
a = 24        # Binary: 11000

result = a >> 2
print(result)  # Output: 6
```

---

e. Assignment Operators

Used to assign values to variables.

Operator	Description	Example
`=`	Assign	`x = 5`
`+=`	Add and assign	`x += 3 → x = x + 3`
`-=`	Subtract and assign	`x -= 3 → x = x - 3`
`*=`	Multiply and assign	`x *= 3 → x = x * 3`
`/=`	Divide and assign	`x /= 3 → x = x / 3`
`//=`	Floor divide and assign	`x //= 3`
`%=`	Modulus and assign	`x %= 3`
`**=`	Exponent and assign	`x **= 3`
`&=`	Bitwise AND and assign	`x &= 3`
`^=`	Bitwise XOR and assign	`x ^= 3`
`<<=`	Left shift and assign	`x <<= 3`
`>>=`	Right shift and assign	`x >>= 3`

Operator Precedence and Associativity

operator precedence determines the order in which different operators are evaluated in an expression, and **associativity** determines the direction in which operators of the same precedence are evaluated (left to right, or right to left).

1. Operator Precedence in Python BEUDA BCLA ELnA

Precedence	Operators	Description	Associativity
1	`()`	Parentheses (used to group expressions)	-
2	`**`	Exponentiation	**Right-to-left**
3	`+x, -x, ~x`	Unary plus, unary minus, bitwise NOT	**Left-to-right**
4	`*, /, //, %`	Multiplication, division, floor division, modulus	**Left-to-right**
5	`+, -`	Addition, subtraction	**Left-to-right**
6	`<<, >>`	Bitwise left shift, bitwise right shift	**Left-to-right**
7	`&`	Bitwise AND	**Left-to-right**
8	`^`	Bitwise XOR	**Left-to-right**
9	`` ` ``	String formatting (backticks in Python 2.x)	**Left-to-right**
10	`==, !=, >, <, >=, <=`	Comparison operators	**Left-to-right**
11	`not`	Logical NOT	**Right-to-left**
12	`and`	Logical AND	**Left-to-right**
13	`or`	Logical OR	**Left-to-right**
14	`=, +=, -=, *=, /=, %=, **=`, etc.	Assignment operators	**Right-to-left**
15	`is, is not, in, not in`	Identity and membership operators	**Left-to-right**

---

2. Associativity in Python

Associativity refers to the direction in which operators of the same precedence are evaluated. There are two types of associativity in Python:

Left-to-Right (LTR) Associativity

Most operators in Python have left-to-right associativity, meaning they are evaluated from left to right.

For example:

```
a = 10 + 5 - 3
```

- First, `10 + 5` is evaluated, resulting in `15`.
- Then, `15 - 3` is evaluated, resulting in `12`.

Right-to-Left (RTL) Associativity

** (exponentiation) operator has **right-to-left** associativity, meaning it is evaluated from right to left.

For example:

```
a = 2 ** 3 ** 2
```

- `3 ** 2` is evaluated first (right-to-left), giving `9`.
- Then, `2 ** 9` is evaluated, giving `512`.

Example 1: `result = 3 + 5 * 2 ** 2 - 10 // 2`

1. **Exponentiation** (`**`) has the highest precedence, so evaluate `2 ** 2` first:
   2**2=4
2. **Multiplication** (`*`) comes next. Now, we evaluate `5 * 4`:
   5*4=20
3. **Floor Division** (`//`) comes after that. Now, evaluate `10 // 2`:
   10//2=5
4. Now, we have:
   `3 + 20 - 5`
5. **Addition** (+) and **Subtraction** (-) have the same precedence, so they are evaluated **left-to-right** (LTR):
   3+20=23
   23−5=18

The final result is `18`.

Example 2: `result = (3 + 5) * (2 ** 2) - (10 // 2)`

1. **Parentheses** override everything, so we first evaluate `(3 + 5)` and `(2 ** 2)`:
   - 3+5=8
   - 2**2=4
2. Now, the expression becomes:
   `8 * 4 - 5`
3. Evaluate **Multiplication** (`*`) first:
   8*4=32
4. Finally, **Subtraction** (-):
   32−5=27

The result is `27`.

Practice Problem

1. Question: What will be the output of the following code?
result = 3 + 5 * 2 ** 2 - 10 // 2 Answer: 18

2. Question: What will this expression return?
result = 6 * 2 // 3 % 4 Answer: 0

3. Question: What will this print?
result = 2 ** 3 ** 2 Answer: 512

4. Question: What will the following code print?
result = 5 + 2 * 3 - 4 ** 2 Answer: -5

5. Question: What is the result of this code?
result = 10 + 5 * (2 ** 3) Answer: 50

6. Question: What will be the result of the following?
result = (10 + 5) * 2 ** 3 // 3 Answer: 40

7. Question: What will be the output of this expression?
result = 5 * 3 - 6 // 3 + 2 Answer: 15

8. Question: What is the result of this code?
result = 3 + 2 * 5 - 4 ** 2 // 2 Answer: 5

9. Question: What will be the result of the following?
result = 2 * 3 + 5 * 2 ** 3 Answer: 46

10. Question: What is the output of this expression?
result = 8 + 2 * (3 ** 2) // 2 Answer: 17
